Building a Graphics Processing Unit (GPU) from scratch in hardware is a massive undertaking. To achieve a functional FPGA prototype in **6 to 7 months**, you must carefully scope the project.

The strategy is to design a **GPGPU (General-Purpose GPU) / Compute Accelerator** rather than a full 3D graphics pipeline. Building rasterizers, texture mapping units, and display engines will stretch your timeline. Focus instead on a **SIMD/SIMT (Single Instruction, Multiple Threads) compute engine** capable of executing simple vector/matrix programs or machine learning kernels.

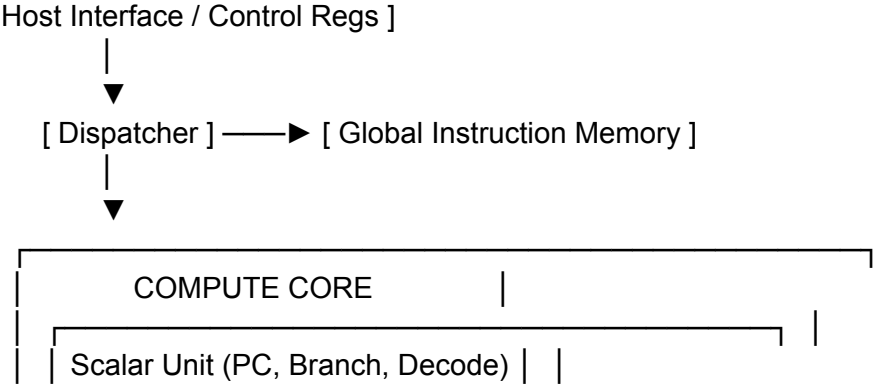
1. Architectural Considerations & Scoping

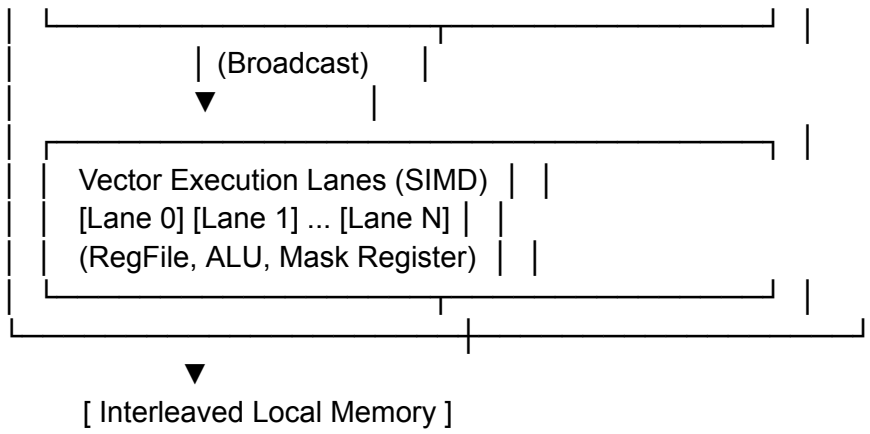
To finish within 7 months, scope your architecture with strict limitations:

- **Execution Model:** SIMT (Single Instruction, Multiple Threads). Design a single "Compute Unit" (CU) or "Core" that executes a **Warp** (e.g., 8, 16, or 32 threads running the same instruction in lockstep).
- **Instruction Set Architecture (ISA): Do not invent your own compiler.** Use a custom, minimal subset of an existing ISA. A highly stripped-down custom variant of **RISC-V** (treating registers as vector lanes) or a minimal custom ISA mapped to an assembler script is ideal.
- **Data Types:** Stick to 32-bit Integers (INT32) or Fixed-Point math. Avoid floating-point arithmetic (FP32) initially, as FPUs consume heavy logic and design time unless using vendor-specific IP blocks.
- **Memory Architecture:** Use on-chip FPGA Block RAM (BRAM). Implement a partitioned memory structure:
    - *Scalar Memory:* For instructions and uniform constants.
    - *Vector/Global Memory:* Interleaved or banked memory with a basic crossbar to handle parallel thread loads/stores.
- **Target Hardware:** A mid-range FPGA development board (e.g., Xilinx Artix-7 A200T or Kintex-7) with enough dual-port BRAMs to act as your register files and memory.

2. Core RTL Components to Build

Your design will be split into three primary structural blocks:





Control & Dispatch

- **Host Interface:** A simple UART, SPI, or AXI-Lite register interface allowing an external CPU to write a kernel to memory, set the thread count, assert a START signal, and poll a DONE status flag.
- **Thread Dispatcher:** Coordinates kernel launches. It calculates work-group offsets and populates thread IDs into the execution lanes.

The Compute Core (SIMT Pipeline)

- **Fetch & Decode Unit:** Fetches one instruction per cycle. Decodes it for both the scalar controller and the parallel vector lanes.
- **Scalar Unit:** Handles control flow (branches, loops) and tracks the program counter (PC).
- **Vector Register File (VRF):** A massive multi-ported or banked memory. For N lanes, each lane needs its own slice of the register file.
- **Execution Lanes (ALUs):** Parallel, identical ALUs that execute operations based on the broadcasted instruction.
- **Execution Mask Unit:** Crucial for SIMT branch divergence. If threads diverge (e.g., an if/else block), this unit tracks an active/inactive bitmask per lane, silencing lanes that shouldn't execute the current instruction.

Memory System

- **Memory Controller & Crossbar:** Routes load/store instructions from individual execution lanes to the banked BRAM data storage, managing bank conflicts.

3. Recommended Reading & Reference Material

Textbook Architecture

- *Computer Architecture: A Quantitative Approach* (Chapters on Data-Level Parallelism and GPUs) by Hennessy and Patterson.
- *Real-Time Rendering* (For understanding graphics pipelines, if you pivot to graphics later).

Open-Source Reference Implementations

- **tiny-gpu (Adam Majmudar):** A minimal GPGPU in Verilog (< 15 files). *Must-read* for understanding how to scope a project of this size.
- **MIAOW GPU (University of Wisconsin-Madison):** An open-source RTL implementation of the AMD Southern Islands ISA. Good for deeper microarchitectural dives.
- **VeriGPU (Hugh Perkins):** A RISC-V-based open-source GPU targeting ML workloads.

4. Verification Plan

Hardware verification will consume roughly **40-50%** of your timeline. Do not write RTL without planning how to test it.

- **Phase 1: Unit Testing (Block Level):** Use **cocotb** (Python-based verification) or SystemVerilog for individual testing of the ALU, Vector Register File, and Branch Mask Unit.
- **Phase 2: Architectural Simulation:** Write a lightweight functional ISA simulator in Python or C++. This acts as your **Golden Model**.
- **Phase 3: Co-Simulation (Differential Testing):** Feed random or structured assembly instructions simultaneously into your RTL simulator (e.g., Icarus Verilog or Verilator) and your Python Golden Model. Compare the register file states after every instruction.
- **Phase 4: Kernel Verification:** Write specific integration kernels (e.g., Vector Addition, Matrix Multiplication) and verify execution from dispatch to the DONE interrupt.

5. 6-7 Month Timeplan & Schedule

Assuming an effort of **15–20 hours/week** for an individual or a small team, here is your roadmap:

Month 1: Specifications, ISA, & Simulation

- **Deliverables:** Written Architecture Specification, Custom ISA definitions, and a functional Python/C++ ISA Simulator.
- **Tasks:** Solidify instruction encoding, register counts, and memory layout. Write your Python assembler to translate mock assembly kernels into binary machine code.

Month 2: Core Pipeline RTL Development

- **Deliverables:** Functional Fetch, Decode, Scalar Control Unit, and execution lanes.
- **Tasks:** Implement the basic 3-to-5 stage execution pipeline. Focus strictly on linear instruction flow (no complex branching yet).

Month 3: SIMT Mechanics & Memory

- **Deliverables:** Execution Mask Unit (Branch Divergence handling) and Banked Local Memory.

- **Tasks:** Code the branch divergence stack logic. Build the memory crossbar interface so vector lanes can read/write data in parallel.

Month 4: Integration, Host Interface, & Unit Testing

- **Deliverables:** Top-level GPU wrapper with Host Command Registers and Core integration.
- **Tasks:** Connect the dispatcher to the core pipeline. Write unit test benches for all sub-modules using cocotb or SystemVerilog.

Month 5: Rigorous Verification (The Bug Hunt)

- **Deliverables:** Co-simulation test suite executing automated verification scripts.
- **Tasks:** Run differential testing against your Golden Model. Debug corner cases, pipeline hazards, stall logic, and memory bank conflicts.

Month 6: FPGA Toolchain & Prototyping

- **Deliverables:** Synthesis-ready code, Vivado/Quartus project configuration, and Timing Closure.
- **Tasks:** Map your generic RTL to FPGA primitives (inferring BRAMs correctly). Run synthesis and implementation. Fix setup/hold timing violations. Connect physical UART/SPI pins for host interaction.

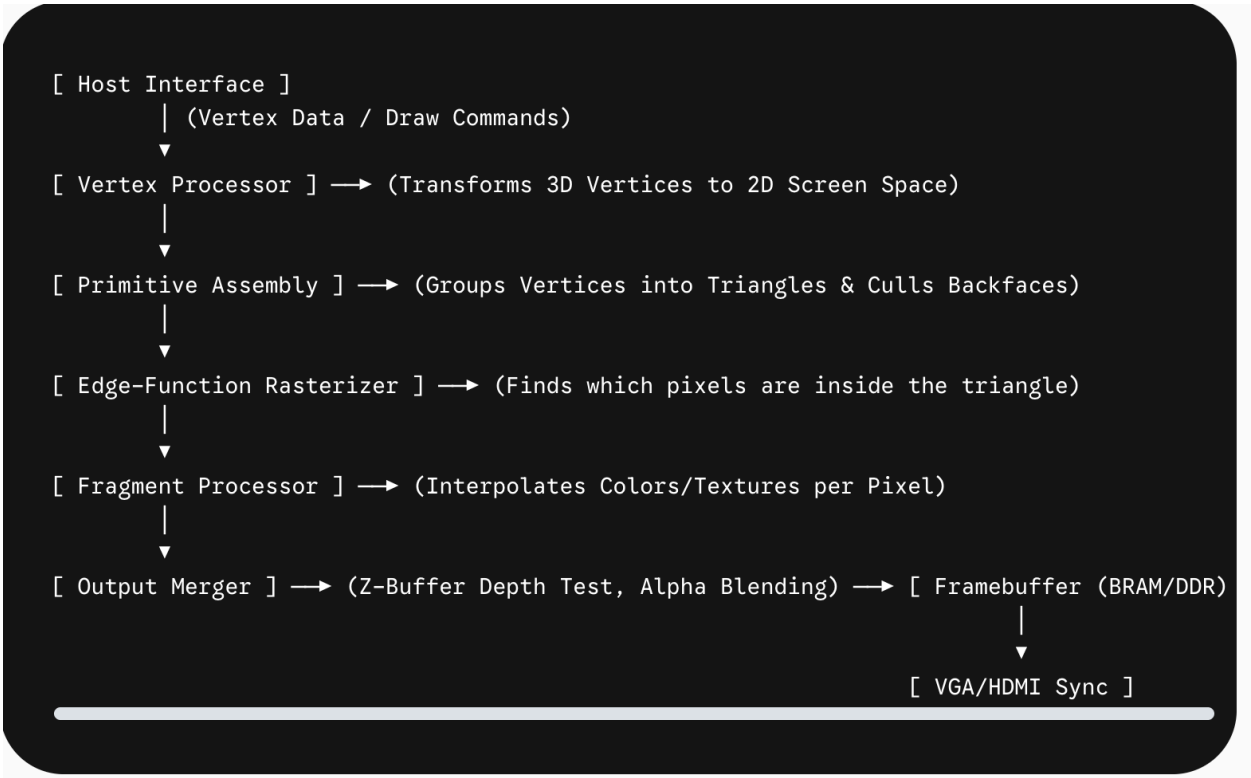
Month 7: Buffer / Hardware Validation (Slack Month)

- **Deliverables:** Working hardware demo executing a live Matrix Multiplication kernel.
- **Tasks:** Use this month to iron out physical system bugs (which always appear), tweak clock speeds, and run your benchmark software stack on the actual FPGA board.

6. Critical Execution Tips

1. **Beware of the Register File Mux:** A vector register file needs many read/write ports. If you don't design it carefully using dual-port BRAMs or block partitioning, the FPGA tools will implement it using Look-Up Tables (LUTs). This will instantly exhaust your FPGA's logic cells and ruin timing.
2. **Start with 4 Lanes:** Do not scale your design to 32 or 64 threads initially. Start with a 4-lane SIMD layout. Scaling up is a matter of changing a parameter (N_LANES) once your memory crossbar is properly parameterized.
3. **Keep Clocks Conservative:** Don't chase a 200 MHz clock rate. Aim for a stable **25 MHz to 50 MHz** on your FPGA prototype. It makes timing closure effortless, allowing you to focus purely on architectural correctness.

Rasterizer:



To bring a rasterizer to life, you need to implement these hardware blocks:

The Rasterizer Engine (The Core Module)

- **Bounding Box Calculator:** Takes the three 2D screen coordinates of a triangle and finds the minimum and maximum X and Y coordinates to limit the pixels you need to check.
- **Edge Function Evaluator:** Uses Pineda's parallel edge function algorithm. For every pixel (X, Y) within the bounding box, it evaluates three linear equations:

$$E_i(X, Y) = (X - X_i) \cdot (Y_{i+1} - Y_i) - (Y - Y_i) \cdot (X_{i+1} - X_i)$$

  If all three edge functions are greater than or equal to zero, the pixel is inside the triangle.

- **Barycentric Coordinate Generator:** Uses the outputs of the edge functions to calculate weights (α, β, γ) for the pixel. These weights are used to linearly interpolate colors, depth (Z), and texture coordinates (U, V) across the triangle face.

Memory & Display Backend

- **Depth (Z) Buffer:** A dedicated block of RAM storing the closest Z value for every pixel. The rasterizer drops pixels that fail the test Znew > Z stored.
- **Color Framebuffer:** Dual-ported BRAM or external DDR memory. While the rasterizer writes pixels to one region (Back Buffer), the display controller reads from another (Front Buffer).
- **VGA/HDMI Controller:** A timing generator that reads the Framebuffer and outputs the exact HSYNC, VSYNC, and RGB signals needed to drive a physical monitor (typically at 640×480 at 60 Hz).

3. Recommended Reading for Rasterization

- **"A Parallel Algorithm for Polygon Rasterization" (Juan Pineda, SIGGRAPH 1988):** *The* foundational paper. It teaches you how to evaluate edge functions incrementally using simple addition instead of expensive multiplications per pixel.
- **Scratchapixel (Online Lessons):** Specifically their sections on "Rasterization: a Practical Implementation." It breaks down the math into incredibly visual, digestible concepts.
- **FPGrows (GitHub project):** A fantastic open-source reference of a tiny, fixed-function hardware rasterizer implemented in Verilog.

4. Adjusted 6-7 Month Timeplan

Because graphics hardware requires a display output and tighter memory sync, the timeline shifts from software-heavy verification to hardware-visual feedback.

Month 1: Math & Fixed-Point Tooling

- **Deliverables:** C/Python fixed-point software rasterizer model.
- **Tasks:** You cannot use floating-point math in your RTL without bloating the design. Master 16.16 fixed-point arithmetic. Write a Python script that takes a 3D .obj file and outputs raw fixed-point hex data for your RTL to read.

Month 2: Display & Framebuffer Logic

- **Deliverables:** Working VGA/HDMI output displaying a static test pattern from BRAM.
- **Tasks:** Get the display timing logic solid immediately. If you can't output a stable image to a monitor early on, debugging your rasterizer later will be impossible.

Month 3: Geometry & Primitive Assembly

- **Deliverables:** Vertex Processor and Backface Culling RTL.
- **Tasks:** Build the hardware that takes vertices, applies basic matrix transformation (or leaves it to the host CPU to pre-transform), and discards triangles facing away from the camera.

Month 4: The Rasterizer Engine

- **Deliverables:** Bounding Box and Pineda Edge Function RTL blocks.
- **Tasks:** Implement the core rasterization loop. Start with a non-pipelined version that checks one pixel per clock cycle to keep it simple.

Month 5: Pixel Pipeline & Z-Buffering

- **Deliverables:** Barycentric Interpolator and Z-Buffer Module.
- **Tasks:** Hook up the color interpolation logic so triangles look smoothly shaded (Gouraud shading) rather than solid blocks. Implement depth testing to handle overlapping geometry.

Month 6: Integration & Simulation

- **Deliverables:** Top-level integration. Simulated image generation.
- **Tasks:** Simulate the whole pipeline by feeding it a single 3D cube. Dump the simulated Framebuffer memory contents into a .ppm image file on your PC to verify it visually before flashing the FPGA.

Month 7: Hardware Testing & Optimization

- **Deliverables:** A real-time, rotating 3D object rendered directly from your FPGA to a monitor.
- **Tasks:** Flash the board. Fix physical memory bottleneck issues. Implement simple double-buffering to eliminate screen tearing.

5. Architectural Trap to Avoid

Do not attempt a "Tile-Based" or deeply parallel rasterizer first. Modern GPUs rasterize blocks of pixels (e.g., 8×8 quads) simultaneously. For your first prototype, build a **Serial Rasterizer** that steps through the bounding box one pixel at a time ($X++, Y++$).

At 640×480 resolution running at 25 MHz or 50 MHz, a serial rasterizer will still easily render thousands of flat-shaded triangles at a smooth 60 frames per second. Keep it simple, get the triangles on the screen, and optimize later!

To get a basic, functional 3D graphics pipeline running on an FPGA within your timeline, you should build a **Fixed-Function Rasterization Pipeline**.

The strategy here is to push the heavy 3D math (like matrix rotations and perspective projection) to the **Host CPU** (your laptop/PC) initially. Your FPGA hardware will focus entirely on what hardware does best: **drawing the triangles rapidly onto a screen.**

Here are the exact hardware blocks you need to write and the architectural flow to connect them.

1. The Core Blocks You Need to Write

You will need to write **6 core RTL blocks**. They are listed here in order of how data flows through them.

Block 1: Host Interface / Register Map (host_interface.v)

- **Purpose:** Receives data from your computer (via UART or SPI) and stores it.
- **Registers needed:**
    - REG_START: Trigger to start drawing a triangle.
    - REG_STATUS: Signals BUSY or READY to the host.
    - REG_V0_X, REG_V0_Y, REG_V1_X...: Fixed-point screen coordinates (X, Y) for the 3 vertices of the triangle.
    - REG_COLOR: The RGB color of the triangle.

Block 2: Triangle Setup Engine (triangle_setup.v)

- **Purpose:** Pre-calculates the constants needed for the Pineda edge functions before rasterization begins.
- **Math it performs:** * Calculates deltas: $dX_0 = X_1 - X_0$, $dY_0 = Y_1 - Y_0$ (repeated for all 3 edges).
    - Calculates the bounding box: Finds the $X_{min}, X_{max}, Y_{min}, Y_{max}$ of the 3 vertices.
    - Calculates the **initial edge function values** (E_0, E_1, E_2) at the starting pixel (X_{min}, Y_{min}).
- **Output:** A SETUP_DONE pulse that triggers the Rasterizer.

Block 3: Bounding Box Rasterizer (rasterizer_core.v)

- **Purpose:** A state machine that loops through every pixel inside the triangle's bounding box and manages the edge accumulators.
- **How it works:** It sets current_x = X_min, current_y = Y_min. It updates the coordinates in a lawnmower pattern (left-to-right, then drops down a row, right-to-left). On every step, it pulses the step signals (step_right or step_down) to the edge blocks.

Block 4: Edge Evaluators (edge_evaluator.v)

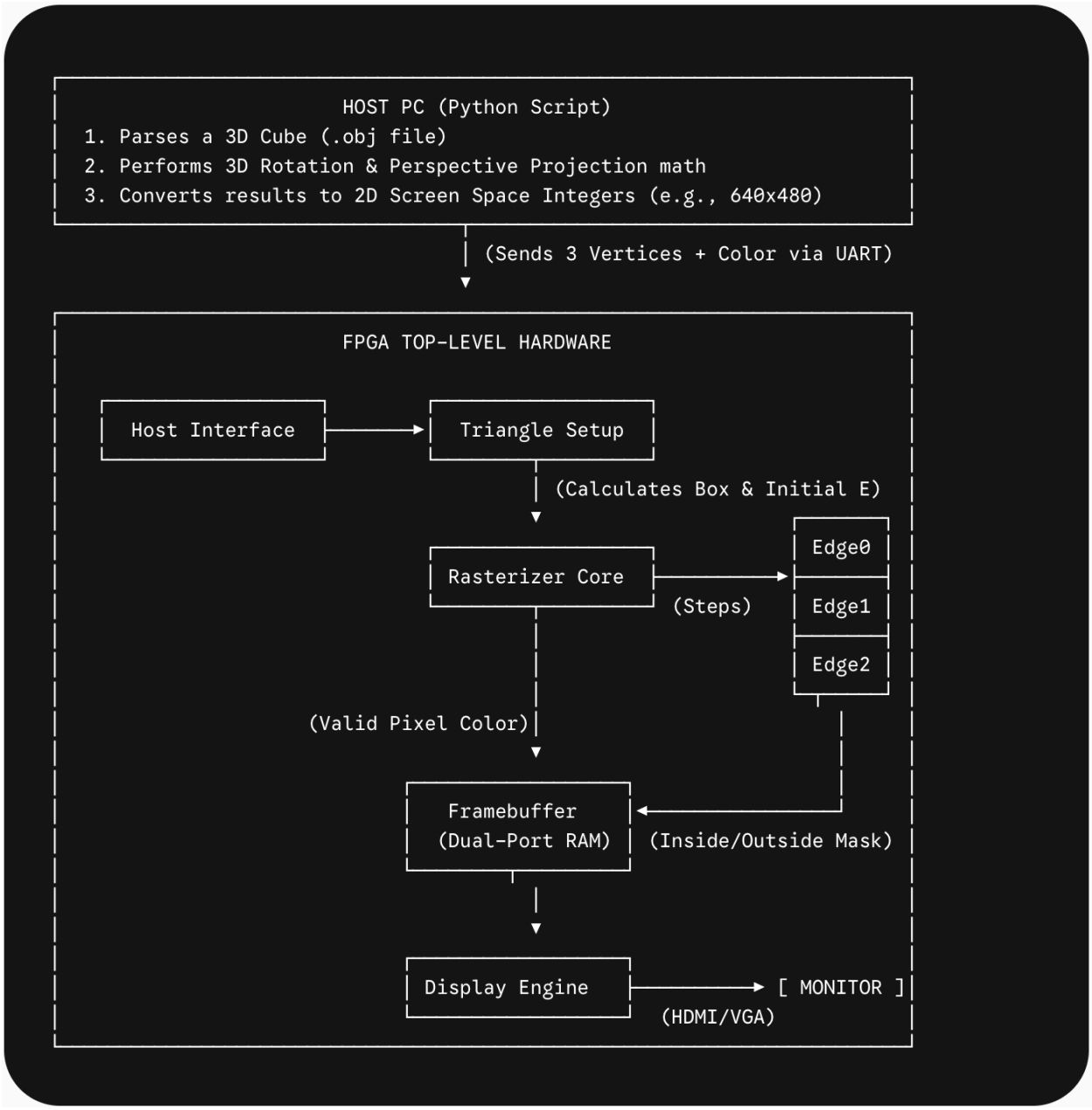
- **Purpose:** Instantiated 3 times inside the Rasterizer. They contain the registers holding the current E values.
- **How it works:** They listen to the step_right or step_down pulses and add/subtract the pre-calculated dY and dX values incrementally.

Block 5: Framebuffer Controller (framebuffer_ctrl.v)

- **Purpose:** Manages the memory where pixels are stored.
- **How it works:** It should map to a dual-port FPGA Block RAM (BRAM).
    - **Port A (Write):** Listens to the Rasterizer. If a pixel is "inside" all 3 edges, Port A writes the REG_COLOR to the memory address calculated by: $\text{Address} = (Y \cdot \text{Screen Width}) + X$.
    - **Port B (Read):** Continuously read by the VGA/HDMI controller to display the image.

Block 6: Display Controller (vga_hdmi_timing.v)

- **Purpose:** Generates the precise pixel clock and synchronization pulses (HSYNC, VSYNC) required to drive a monitor.
- **How it works:** It constantly sweeps from $(0,0)$ to $(640,480)$. As it sweeps, it requests the color data from Port B of the Framebuffer and outputs it to the physical FPGA pins.


HOST PC (Python Script)
1. Parses a 3D Cube (.obj file)
2. Performs 3D Rotation & Perspective Projection math
3. Converts results to 2D Screen Space Integers (e.g., 640x480)
(Sends 3 Vertices + Color via UART)
FPGA TOP-LEVEL HARDWARE
Host Interface
Triangle Setup
(Calculates Box & Initial E)
Edge0
Rasterizer Core
(Steps)
Edge1
Edge2
(Valid Pixel Color)
Framebuffer
(Dual-Port RAM)
(Inside/Outside Mask)
Display Engine
[MONITOR]
(HDMI/VGA)

3. How to Try It: A Step-by-Step Execution Plan

To guarantee success without drowning in complexity, build this in **three progressive stages**:

Stage 1: The "Donut" Test (Display & Framebuffer)

Don't write a rasterizer yet.

1. Write **Block 6 (Display Controller)** and **Block 5 (Framebuffer)**.
2. Write a simple piece of hardcoded Verilog logic that pre-fills the Framebuffer memory with a static image (like a colorful square or circle).
3. Flash the FPGA and make sure you can see that square perfectly on your monitor. *If your display timing is wrong, you can't debug anything else.*

Stage 2: The "Software Simulation" Test (The Sandbox)

Before touching FPGA hardware for the rasterizer, build it in software to prove your math works.

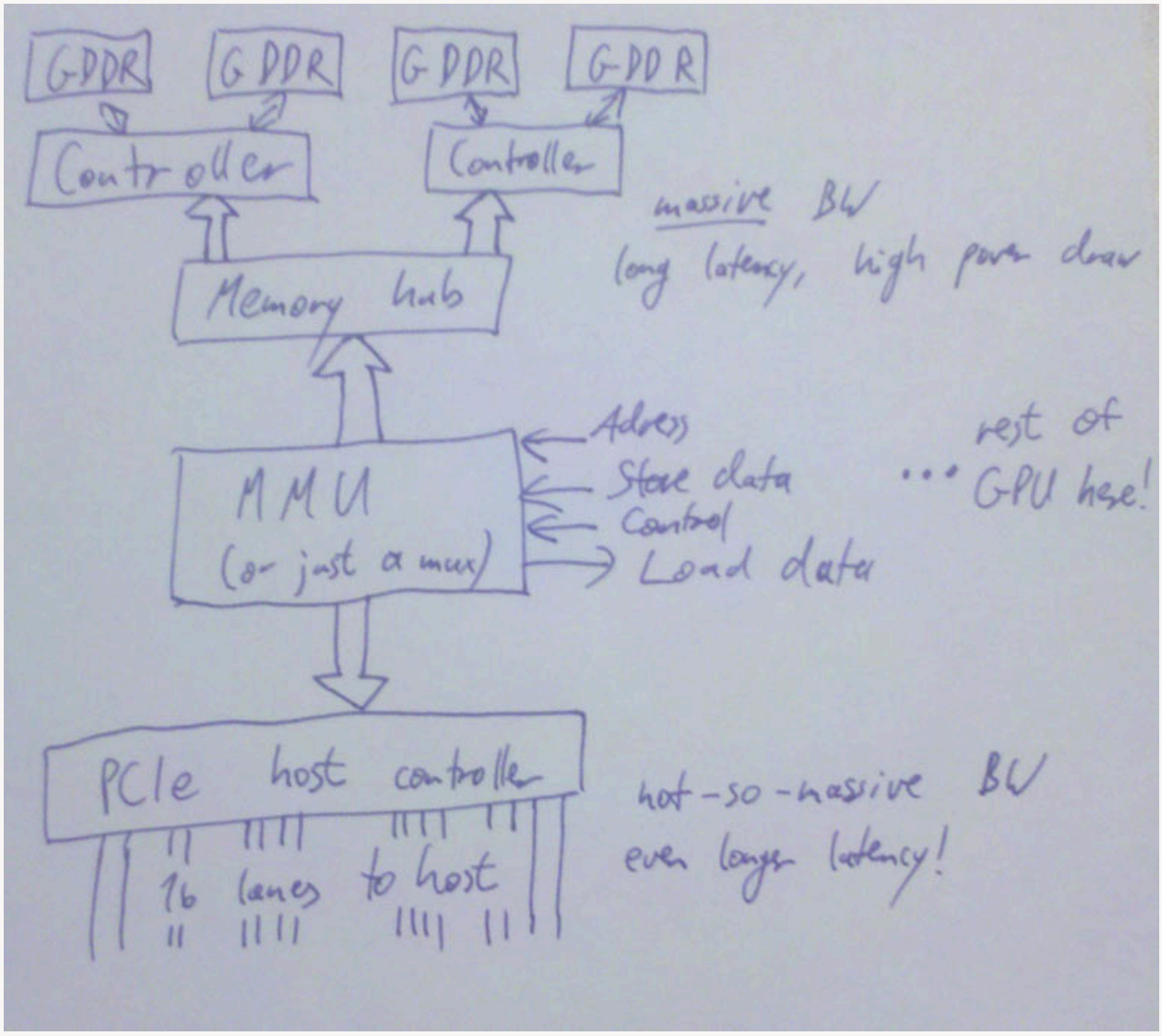
1. Write a quick Python script that implements Pineda's algorithm using the exact same stepped logic ($E + dY$, $E - dX$) your hardware will use.
2. Have the script output a tiny 64×64 pixel text grid using coordinates like * for inside and . for outside.
3. Once your Python script can successfully draw a triangle in text, you have the "golden design" for your RTL modules.

Stage 3: The Single Triangle Test (Hardware Integration)

1. Write **Blocks 2, 3, and 4** (Setup, Rasterizer, Edges) in Verilog.
2. Connect them to the Framebuffer you built in Stage 1.
3. Use your **Host Interface (Block 1)** to manually send the coordinates for a single, giant triangle: $V_0(100, 100)$, $V_1(300, 100)$, $V_2(200, 300)$.
4. Press "Start". If everything is wired correctly, the rasterizer will sweep the bounding box, find the inside pixels, write them to the Framebuffer, and a clean triangle will instantly appear on your monitor.

GPU –

1. A GeForce GTX 480, on the other hand, has a total memory bandwidth of close to 180 GB/s..
2. https://fgiesen.wordpress.com/2011/07/02/a-trip-through-the-graphics-pipeline-2011-part-2/
3.



3.

`IA` — Input Assembler. Reads index and vertex data.
`VS` — Vertex shader. Gets input vertex data, writes out processed vertex data for the next stage.
`PA` — Primitive Assembly. Reads the vertices that make up a primitive and passes them on.
`HS` — Hull shader; accepts patch primitives, writes transformed (or not) patch control points, inputs for the domain shader, plus some extra data that drives tessellation.
`TS` — Tessellator stage. Creates vertices and connectivity for tessellated lines or triangles.

`DS` — Domain shader; takes shaded control points, extra data from HS and tessellated positions from TS and turns them into vertices again.
`GS` — Geometry shader; inputs primitives, optionally with adjacency information, then outputs different primitives. Also the primary hub for…
`SO` — Stream-out. Writes GS output (i.e. transformed primitives) to a buffer in memory.
`RS` — Rasterizer. Rasterizes primitives.
`PS` — Pixel shader. Gets interpolated vertex data, outputs pixel colors. Can also write to UAVs (unordered access views).
`OM` — Output merger. Gets shaded pixels from PS, does alpha blending and writes them back to the backbuffer.
`CS` — Compute shader. In its own pipeline all by itself. Only input is constant buffers+thread ID; can write to buffers and UAVs.

Input Assembler (IA) + Vertex Shader (VS) -> geometry_transform.sv

- **What it reads:** The IA reads a small array of 3D coordinates (X, Y, Z) and colors (R, G, B) out of an internal setup memory.
- **The "Shader" reality:** Instead of a programmable core, your VS is a **Fixed Matrix Multiplier**. You write sequential multiply-accumulate (MAC) blocks that multiply the incoming 3D vertex by a 4×4 View-Projection Matrix (loaded into your Host Interface registers) to project the 3D points onto a 2D screen space.

2. Primitive Assembly (PA) -> primitive_assembly.sv

- **What it does:** It gathers groups of 3 processed vertices from the VS.
- **The hardware check:** It calculates the cross-product of the triangle's screen vectors. If the sign is negative, the triangle is facing away from the camera (**Backface Culling**). The PA instantly drops the triangle, saving downstream rasterizer clock cycles.

3. Rasterizer (RS) + Pixel Shader (PS) -> rasterizer_pixel_pipeline.sv

- **The Rasterizer (RS):** This is your Pineda loop engine. It scans the bounding box of the triangle passed by the PA and evaluates the edge equations.
- **The Pixel Shader (PS):** For every pixel where the RS says "Inside", the PS determines its appearance. To keep it simple, your first hardware PS will just calculate **Barycentric Interpolation** to smoothly blend the vertex colors across the face of the triangle (Gouraud Shading).

4. Output Merger (OM) -> output_merger.sv

- **The gatekeeper:** It receives the target (X,Y) coordinate, the interpolated Z (depth), and the color from the PS.
- **The hardware check:** It reads the current depth value from the Z-Buffer RAM at address $(Y \cdot \text{Width}) + X$. If $Z_{new} < Z_{old}$, it passes the color to the Framebuffer and updates the Z-Buffer with Z_{new}.

4. Where does the Compute Shader (CS) fit?

As you noted, the **Compute Shader (CS)** bypasses the graphics rendering pipeline completely. If you want a unified GPU that can handle *both* graphics and compute within 7 months:

1. Build the **Graphics Pipeline** (IA $\rightarrow$ VS $\rightarrow$ PA $\rightarrow$ RS $\rightarrow$ PS $\rightarrow$ OM) as described above first.
2. If you have time left over in Month 6, you can implement a separate, tiny **SIMD Vector Execution Unit** (the CS) that sits parallel to the graphics pipeline, sharing access to the master Host Interface and the Framebuffer BRAM via an AXI arbiter.

Rule of thumb for a 7-month solo build: **Pick one lane first.** Build the fixed-function graphics pipeline to get stunning visual output, or focus purely on a CS-style compute accelerator for matrix math. Trying to design both hardware tracks simultaneously will bust your timeline.

Github Code:

https://github.com/Humblefool14/leetcode_gpu

1. Set of goals.
2. Decide Architecture/scope of work for 6-7 months.
3. The ip's, performance, io's, threads, why we need these specific units, tiling requirements, transforms from one buses to another, profiling for area/power/node,etc, details,
4. AHB/AXI to APB, Arbiters, Fifo's, Pointer Fifo's, Async fifo's, Formal Verifications for modules,
5. What are the requests for ML/Training/Inference or Plain Graphics ?
6. Debugability features that we want ? Jtag/Interrupts/counters/fsm ?
7. https://www.youtube.com/watch?v=t7Ztio8cwqM
8. Brenshan Algorithm
9. Brenshan's algorithm dry run: https://www.youtube.com/watch?v=oHuxM83W8C8
10. https://www.youtube.com/watch?v=WRqOO8UFwOU&list=PLNhdAzzsGnqN5rx1wkKrugLHp-N6GcjmT
11. GIThub : https://github.com/Humblefool14/leetcode_gpu
12.